ព្រះរាជាណាចក្រកម្ពុជា

ជាតិ សាសនា ព្រះមហាក្សត្រ

REPORT FOR OOP

TP -Lab1

Name of Student	ID	Group
YET VAYU	e20240092	I3-AMS(D)

Lecturer: **Mr. VANN Visal**

Academic Year: 2026-2027

- ❖ Python Basic data types
    - str Text value, for example "Eliza".
    - int : Whole number, for example 20.
    - **Float :** Decimal number, for example 3.5.
    - **Bool : True or False.**
- ❖ **Collection and references types**

Category	Data Type	Syntax / Bracket	Key Characteristics
Primitive	**str, int, float, bool**	**Direct literal**	**Atomic scalar values representing text, integers, decimals, or truth values.**
List	**list**	**[ ]**	**Ordered, mutable collection; allows duplicate elements.**
Tuple	**tuple**	**( )**	**Ordered, immutable collection; fixed integrity.**
Set	**set**	**{ }**	**Unordered collection of unique values; no duplicates.**
Dictionary	**dict**	**{key: value}**	**Key-value mappings; unique keys route to arbitrary values.**

- ❖ Class and Object

A class is a blueprint; an object is an instance containing specific attribute values.

- ❖ Python literals

Literals are values written directly in source code: numbers, text, booleans, None,

And The type() function confirms the data type of every literal value.

- ❖ Local variables

A temporary variable created inside a function or method. It exists only while that function executes and cannot be accessed outside its scope.

- ❖ **Instance Attributes**

Variables attached to individual objects via self inside __init__(). Each object created from the class maintains its own unique state. Constructor (`__init__`) : Runs automatically to set up initial data whenever an object is created.

Attribute / Variable Type	Declaration Scope	Lifecycle & Scope Boundary
Local Variable	Inside function / method body	Created on execution; destroyed upon function exit. Access outside raises NameError.
Instance Attribute	Inside __init__() via self	Unique per object instance; stored in the object's instance dictionary (__dict__).
Class Attribute	Directly inside class body	Shared equally across all instances of the class blueprint; exists in class

- Class attributes

Variables defined directly inside the class block (outside methods). They belong to the blueprint itself and are shared equally by all instances.

Class-Level Declaration : Declared directly in class scope without using `self.`

Shared Memory Space : All objects read the exact same value from the class blueprint.

- **Public naming convention** : can access from anywhere inside or outside the class.It usually no prefix(self.name)
- **Non-public naming convention :** Meant for internal use within the class and its subclasses (though not strictly enforced) and Single leading underscore _ (e.g., self._name)
- **Private naming convention** : is a set of rules for naming variables and methods intended for internal use rather than public access.(double underscore__)

Access Type	Naming Syntax	Behavior & Access Scope
Public	self.price	Accessible from anywhere inside or outside the class.
Non-Public (Protected)	self._stock	Convention indicating internal use; still accessible externally.
Private	self.__balance	Triggers name mangling to _ClassName__balance to prevent collision.

- **Access Control by Convention** means using naming rules to indicate how attributes and methods should be accessed in a Python class.
- **A @staticmethod is a method** defined inside a class that does not automatically receive the instance (self) or the class (cls) as its first argument**.**
- **@classmethod decorator** defines a method that receives the class as its first argument, conventionally named cls. It is useful for working with class-level attributes and creating alternative ways to construct objects.

Feature	Decorator / Import	Purpose & Runtime Behavior
Abstract Class	from abc import ABC	Blueprint class that cannot be instantiated directly if abstract methods exist.
Abstract Method	@abstractmethod	Forces child subclasses to provide concrete method implementat
Property Manager	@property, @x.setter	Allows method calls to behave like attribute access while adding validation.
Final Constraint	@final (typing)	Signals static type checkers that a method or class cannot be overridden.

- **Python Operators :** Python provides robust operators for arithmetic calculations, relational checks, binary bit manipulations, and logical condition chaining.

Operator Category	Symbols / Keywords	Core Description
Arithmetic	+, -, *, /, //, %, **	Standard calculations, floor division, modulo remainders, exponentiation.
Comparison	==, !=, >, <, >=, <=	Relational checks comparing values returning Booleans (True/False).
Bitwise	&, |, ^, ~, <<, >>	Binary-level bit manipulations, XOR, inversion, and shifts.
Logical	and, or, not	Condition chaining and Boolean algebra evaluation.

❖ **Bitwise Shift Rule**

Left shift (<< n) multiplies an integer by 2 raised to power *n*, while right shift (>> n) floor-divides the integer by 2 raised to power *n*.

- And return true if both conditions are correct.
- Or return true if at least one condition is true.
- Not reverse the conditions of the Booleans value conditions

Special Operator Group	Operators	Behavior & Purpose
Assignment	+=, -=, *=, /=, //=, **=	In-place arithmetic updates (e.g., x += 5 expands to x = x + 5).
Identity	is, is not	Checks if two names reference the exact same memory object (unlike ==).
Membership	in, not in	Validates if an element exists inside a collection (list, tuple, set, dict).
Conditional	x if condition else y	One-line ternary expression returning *x* if condition holds, else *y*.

Precedence Hierarchy

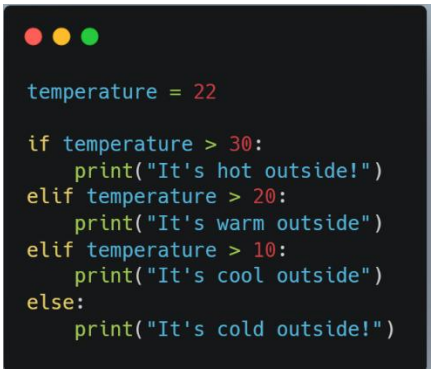
Operator evaluation priority follows: Exponentiation (**) -> Multiplication/Division -> Addition/Subtraction -> Comparisons -> Logical. Use parentheses () to explicitly override precedence.

❖

❖ **CORE DECISION STRUCTURES**

- **if Statement**

Executes block only when condition evaluates to True.



- **if-else Statement**

Provides two paths: one for True and one for False.

- **if-elif-else**

Evaluates multiple sequential conditions in order.

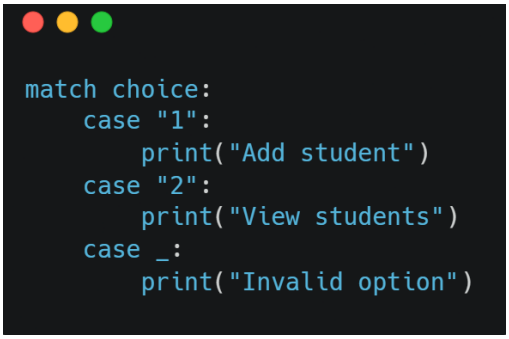
OUTPUT: **"It's warm outside"**.

❖ **Nested If & Match-Case**

- **Nested if statement:** Evaluates inner conditions only when outer condition passes.



- **Match case pattern:** Replaces long if-elif chains with structural pattern matching.



- **One-Line Value Selection**

Selects one of two values in a single concise expression. Clean alternative to short if-else blocks.

  - **Python Syntax:** value1 if condition else value2
  - **Java Equivalent:** condition ? value1 : value2

```
# Assign status based on score
score = 65

result = "Pass" if score >= 50 else "Fail"

print(result)
# Output: Pass
```

❖ **While vs For Loops**

- **While loop:** Repeats as long as a condition stays True. Checks condition before each iteration.

```
count = 1
while count <= 3:
    print("Round", count)
    count += 1
```

- **For loop:** Iterates directly over sequence items (lists, strings, range). Auto-increments counter.

```
for fruit in ["apple", "mango"]:
    print(fruit)

for i in range(1, 4):
    print(i)
```

❖ **break, continue & pass**

- The **break** statement in Python is a loop control keyword used to immediately terminate the nearest enclosing for or while loop.

- the **continue** statement is a loop control mechanism that **skips the rest of the current iteration** and immediately proceeds to the next iteration of the loop.
- The **pass** statement in Python is a null operation that does nothing when executed, serving primarily as a **syntactic placeholder** to prevent errors in empty code blocks.

❖ PYTHON VS JAVA SYNTAX

Feature	Python	Java
Code Blocks	Indentation (Whitespace)	Curly braces { }
For Loop Structure	for x in items:	for (int i=0; i<n; i++)
do-while Loop	Not available (while True + break)	Native do { } while();
Statement Endings	Newline (No semicolon)	Semicolon ; required